



RoboCorea — Operations runbook (mapping, localization & navigation)

Audience: someone who knows the robot and what it can do, but has never run its software. This is the *practical* companion to `reference/architecture.md` (the design source of truth). If a command here disagrees with the architecture doc, trust this file for *how to run it* and the architecture doc for *why*.

Everything below is the **2-D SLAM → localization → Nav2** stack and the one-command bring-up. The arm, video and teleop are only mentioned where they intersect that flow.

The two machines

Machine	Role	Static IP	Bring-up
Jetson (Orin Nano, on the robot)	sensor + relay + perception	192.168.50.10	<code>./jetson.sh</code>
Laptop (operator workstation)	GUI + arm + the human with the RC TX	192.168.50.11	<code>./laptop.sh</code>

They talk over **Zenoh** (`rmw_zenoh`) on a private Wi-Fi/Ethernet link, partition `ROS_DOMAIN_ID=20` . Video (C920) rides **SRT**, not ROS. The **RC transmitter** (FlySky) is the low-latency teleop link and is independent of the network — it works even if ROS is down.

The whole flow in one breath: boot both machines → `./jetson.sh` on the robot, `./laptop.sh` on the laptop → in the GUI **Robot Systems** window start **Sensors**, then **Mapping** → drive the arena on the RC to build the map → **Save Map** → later, **Load** that map → **Localization** → set the start pose → **Navigation** → enable **AUTO DRIVE** and send a goal.

Part 0 — First-time setup (once per machine, or after re-imaging)

Skip to [Part 1](#) if both machines were already set up and you are just rebooting.

0.1 Network (both machines)

Static IPs over the shared link (NetworkManager, `ipv4.method manual`):

Jetson `192.168.50.10` , laptop `192.168.50.11` . Confirm they can ping each other:

```
ping -c2 192.168.50.10    # from the laptop
ping -c2 192.168.50.11    # from the Jetson
```

0.2 Zenoh middleware (once per machine — required)

This makes `rmw_zenoh` the default ROS middleware and installs an always-on Zenoh router (a system service that starts at boot). Run it **once on each machine**, pointing each at the *other* machine's IP:

```
# on the JETSON (peer = the laptop):
sudo PEER_IP=192.168.50.11 bash scripts/setup-zenoh.sh
# on the LAPTOP (peer = the Jetson):
sudo PEER_IP=192.168.50.10 bash scripts/setup-zenoh.sh
```

After this, every ROS process on the host uses Zenoh, and the routers federate discovery over a single TCP link. Verify:

```
systemctl status rmw-zenoh-router.service    # active (running)
echo "$RMW_IMPLEMENTATION"                  # rmw_zenoh_cpp (in a NEW shell)
```

`setup-zenoh.sh` is **idempotent** — safe to re-run to re-point the peer or after re-imaging. If you change `ROS_DOMAIN_ID` , re-run it with the new value so the router's domain matches.

0.3 Stable serial device names (Jetson — strongly recommended)

The robot has three USB-serial devices whose `/dev/tty*` numbers can race at boot. The **bridge auto-binds the two ESP32 boards by identity**, so it does not need a fixed name — but the **RPLidar** does (`lidar.service` opens `/dev/rplidar`).

- **RPLidar A2M12** (CP2102 → `tttyUSB*`): install the udev rule that gives it the stable `/dev/rplidar` symlink:

```
cd software/ros2_ws
sudo cp src/sllidar_ros2/scripts/rplidar.rules /etc/udev/rules.d/
sudo udevadm control --reload-rules && sudo udevadm trigger
ls -l /dev/rplidar          # -> ../tttyUSBn
```

- **ESP32 boards** (CH340): on the Jetson's stock kernel these enumerate as `/dev/ttyCH341USB*` (the out-of-tree WCH driver), **not** `tttyUSB` . The bridge's default scan list already includes `/dev/ttyCH341USB*` , so no rule is needed. If you want to pin one explicitly, use `./jetson.sh --serial /dev/ttyCH341USB0` .

0.4 Build the workspace (both machines)

```
cd software/ros2_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install
```

...or just let the bring-up scripts do it the first time: `./jetson.sh --build` (robot) / `./laptop.sh --build` (laptop). After this, the scripts install the Jetson's systemd units and enable headless persistence automatically.

Part 1 — Every boot: bring it all up

After a fresh reboot of both machines:

On the Jetson (robot)

```
cd ~/RoboCorea
./jetson.sh
```

This brings up, as managed services that survive an SSH drop / reboot:

- **esp32-bridge** — the serial↔ROS relay (drive, flippers, arm, thermal+mag, wheel odom). The core link.
- **robot-manager + map-manager** — the services the GUI's Robot Systems window calls (start/stop perception stacks; save/load named maps).
- **c920-stream** — the C920 video over SRT to the GUI.

It leaves **ZED + Lidar + SLAM/Localization/Nav** *off* — you start those from the GUI when you want them (or pass `--sensors` to start sensors + SLAM immediately).

Check it: `./jetson.sh --status`. You should see the four services *active* and `/robot/*` services on the bus.

On the laptop (operator)

```
cd ~/RoboCorea
./laptop.sh
```

This launches the **GUI** (foreground; Ctrl-C to quit) plus the digital twin, the arm servo, and gamepad arm teleop. Once it's up you should see live video, telemetry on the dashboard, and — when the robot is reachable — a green connection LED.

If the dashboard shows no telemetry: the robot and laptop aren't talking. See [Troubleshooting → Robot not visible](#).

That's the whole bring-up. The rest of this runbook is *what you do in the GUI*.

Part 2 — Build a map while teleoping, then save it

You drive the robot manually (RC) while SLAM builds the 2-D map from the lidar + fused odometry. Open the GUI **Robot Systems** window (the  toolbar button next to ).

1. **Robot Systems → Perception → Sensors:** click **Start ZED+Lidar**. Wait for the LED to go green (*active*). This starts the ZED2 + RPLidar drivers on the robot.
2. **Perception → Mapping / SLAM:** click **Start SLAM**. This starts `slam_toolbox` + the EKF + the shared sensor/odom front-end on the robot.
3. Click **Open 2D Map**. A live map window opens (the grid fills in as the robot sees the arena; the robot icon is its `map-base` pose).
4. **Drive the arena on the RC transmitter** to cover the whole track:
 - **Ch3** = forward/back, **Ch4** = turn. **Ch2** = flipper rate, with **Ch1/Ch5** selecting which flipper(s). **Ch6 down = E-STOP**, up = virtual-flip ("drive from the other end").
 - Drive **smoothly** and revisit places — that lets SLAM close loops and keeps the map crisp. Avoid spinning in place fast (bad for visual odometry).
 - Watch the map window: walls should stay sharp and not "double up". If the map smears, slow down / back up to a known spot to let it re-register.
5. When the arena is fully covered, **save the map** without stopping SLAM: in the **2D Map window** click **Save Map...**, type a name (e.g. `arena1`), confirm. It grabs a preview thumbnail and saves to the robot.

That's it — the map now lives **on the Jetson** under `~/maps/arena1/` (`occ.pgm` + `occ.yaml` for Nav2/AMCL, `map.posegraph` + `map.data` for `slam_toolbox`). It **persists across reboots** — it's on disk, not in RAM.

Keeping / managing maps

- The **Maps** tab in the Robot Systems window lists every saved map with 2D/3D badges and thumbnails. **Refresh** re-reads the library; **Delete** removes one.
- To back a map up off the robot (so a re-image can't lose it), from the laptop:

```
scp -r susana-akemi@192.168.50.10:~/maps/arena1 ~/map-backups/
```

CLI equivalents (when the GUI isn't available)

```
# start sensors + SLAM on the robot without the GUI:
./jetson.sh --sensors
# ...drive the arena on the RC...
# save (run on the Jetson, or from the laptop – same Zenoh graph):
ros2 service call /robot/maps/save rescue_interfaces/srv/SaveMap "{name: arena1, kind: '2D'}"
ros2 service call /robot/maps/list rescue_interfaces/srv/ListMaps "{}"
```

Part 3 — Use a saved map: localize and navigate

Now the map exists, you can relocalize on it and let Nav2 drive. In the GUI

Robot Systems window:

"I just mapped, saved, and stopped Mapping in this same session — do I have to Load the map again?" — Yes. Mapping and localization are **two different**

stacks: while mapping, `slam_toolbox` ran in *mapping* mode and owned `map→odom + /map`. Stopping the Mapping stack tears all of that down. The Maps tab's **Load** is not just a file picker — it (re)starts the **Localization** stack (AMCL + `map_server`) on the saved map. So even with the map fresh on disk you still: **Stop Mapping → Maps tab → Load** (Refresh first if it's not listed yet) → **Set Start Pose** at the robot's current spot (it's wherever it finished mapping) → **wait for preflight green → Start Nav**. You do **not** re-drive or re-map. Skip straight to step 2 below — sensors are already running from mapping.

Shortcut (no save/reload): you can also **Start Navigation directly on top of a still-running Mapping stack** — Nav2 will drive the live map (online SLAM, like the sim demo). It's faster but the map keeps changing under you, and the GUI pre-flight gate stays grey (it runs with the *Localization* stack, not Mapping), so you confirm through the "Start Nav2 anyway?" prompt. Use the Stop→Load path above for a stable competition map.

1. **Perception → Sensors → Start ZED+Lidar** (localization needs `/scan` + ZED odom). You do **not** need the Mapping stack for this. (*If you're continuing*

straight from mapping, sensors are already up — go to step 2.)

2. **Maps tab:** select your map (e.g. `arena1`) → **Load** (choose *2D* or *both*). This (re)starts the **Localization** stack on the robot with that map (AMCL + `map_server` + EKF + the navigation pre-flight check). The Perception tab's **Localization** LED goes green.
3. **Set the start pose.** Open the **2D Map**, click **Set Start Pose**, then **click-drag** on the map at the robot's real location: the click sets X/Y, the drag direction sets heading. This publishes `/initialpose`; AMCL's particle cloud should tighten around the robot.
4. **Wait for pre-flight to go green.** Under the **Navigation** stack the `preflight:` line reports `scan / odom / tf / map`. All four must pass before navigation will start. (`blocked` = something's missing — usually you haven't loaded a map or set the start pose.)
5. **Perception → Navigation (Nav2) → Start Nav.** Nav2 (planner + controller + behaviors) comes up on the robot. If pre-flight isn't green it warns first.
6. **Let it actually drive.** Nav2 only *plans* — it publishes `/cmd_vel` but the wheels stay in RC mode until you flip the dashboard **AUTO DRIVE** toggle on. This is the autonomy gate. With it on:
 - send a goal (a **Nav2 Goal** in RViz/the map, `ros2 run rescue_nav waypoint_runner`, or `ros2 topic pub --once /goal_pose ...`), and the robot drives the planned path.
7. **Safety handoff (always live):** nudging an **RC drive stick** or engaging **virtual-flip** instantly **latches AUTO DRIVE off** and returns the tracks to you — re-enabling autonomy is then a deliberate GUI action. **E-stop** (Ch6 down, or the GUI software e-stop) stops everything regardless.

CLI equivalent

```
# load + localize on a saved map (run on the robot or laptop):
ros2 service call /robot/maps/load rescue_interfaces/srv/LoadMap "{name: arena1, kind: '2D'}"
# set the start pose, e.g. at origin facing +x:
ros2 topic pub --once /initialpose geometry_msgs/msg/PoseWithCovarianceStamped \
  "{header: {frame_id: 'map'}, pose: {pose: {position: {x: 0.0, y: 0.0, z: 0.0}, orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}}}"
# start Nav2:
ros2 service call /robot/navigation/start std_srvs/srv/Trigger "{}"
# enable autonomy drive + send a goal (GUI AUTO DRIVE toggle, or):
ros2 topic pub --once /autonomy/enable std_msgs/msg/Bool "{data: true}"
ros2 topic pub --once /goal_pose geometry_msgs/msg/PoseStamped \
  "{header: {frame_id: 'map'}, pose: {position: {x: 2.0, y: 0.0, z: 0.0}, orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}}}"
```

Part 4 — Parameters & overrides

Everything works on sensible defaults; reach for these only when you need to.

jetson.sh flags

Flag	Effect
--build	colcon build the Jetson packages before bringing up.
--sensors	Also start ZED+Lidar + SLAM immediately (else GUI-on-demand).
--serial <dev>	Pin the bridge to one ESP32 device, e.g. --serial /dev/ttyCH341USB0 (custom PCB port). Omit it and the bridge scans ttyCH341USB* + serial/by-id .
--no-ekf	Bench / handheld odometry: use ZED-only planar odom, skip the EKF fusion. Use when there are no tracks/bridge or you're carrying the robot — the wheel-fusion EKF mis-models that. Prod (default) fuses wheel + ZED VIO + ZED IMU.
--no-camera	Don't start the C920 SRT streamer.

Flag	Effect
<code>--no-bridge</code>	Managers only (no esp32-bridge) — rarely needed.
<code>--domain N</code>	<code>ROS_DOMAIN_ID</code> (default 20). Must match the laptop and the Zenoh router (re-run <code>setup-zenoh.sh</code> with the new domain).
<code>--status</code> / <code>--stop</code>	Show running services / stop everything this script starts.

The **odometry mode** is also live-switchable without restarting the script (the GUI Start-SLAM button picks it up):

```
systemctl --user set-environment ROBOCOREA_USE_EKF=false # bench (ZED-only)
systemctl --user unset-environment ROBOCOREA_USE_EKF    # back to prod (EKF)
```

`laptop.sh` flags

Flag	Effect
<code>--build</code>	<code>colcon build</code> the workstation packages first.
<code>--no-joystick</code>	No gamepad arm teleop (keyboard/RViz only).
<code>--rviz</code>	Also open RViz with the digital-twin config.
<code>--domain N</code>	<code>ROS_DOMAIN_ID</code> (default 20; must match the robot).

Autonomy drive gate (recap)

The base is RC-only by default. Nav2's `/cmd_vel` reaches the tracks **only** when **AUTO DRIVE** is on *and* the RC drive sticks are neutral *and* the command is fresh. Default is **off**. See architecture §16.

Part 5 — When the Jetson is powered back on (what to verify / change)

The robot was off while this runbook was written; once it's reachable again

(`ssh susana-akemi@jaguar.local` , or `...@192.168.50.10`), check these and adjust the marked files **if the real values differ**:

1. **Network + Zenoh**: confirm the static IP (`ip a` → `192.168.50.10`) and run `sudo PEER_IP=192.168.50.11 bash scripts/setup-zenoh.sh` if it hasn't been run on this image. Then a fresh shell should show `RMW_IMPLEMENTATION=rmw_zenoh_cpp` .
2. **Serial devices** (`ls -l /dev/serial/by-id/ /dev/ttyCH341USB* /dev/ttyUSB*`):
 - If the ESP32 boards do **not** appear as `ttyCH341USB*` , add their glob to the bridge's `serial_candidates` (in `software/ros2_ws/src/esp32_bridge/launch/esp32_bridge.launch.py`), or just pass `./jetson.sh --serial <dev>` .
 - Install the **RPLidar udev rule** ([§0.3](#)) so `/dev/rplidar` exists — `lidar.service` opens that exact path.
3. **Camera devices** (`v4l2-ctl --list-devices`): the C920 streamer is hardcoded to `/dev/video2` (port 8890). If the C920 enumerates elsewhere, edit the `CAMERAS=(...)` array at the top of `software/ros2_ws/src/gui/scripts/c920_srt_stream.sh` .
4. **ZED model**: `zed.service` launches `camera_model:=zed2` . If it's a ZED (1) or ZED Mini, edit that in `software/ros2_ws/src/rescue_bringup/systemd/zed.service` (only the model/calibration changes; the `/zed/*` topics stay the same).
5. **Workspace path**: the systemd units assume the checkout is at `~/RoboCorea` . If it's elsewhere, fix the `%h/RoboCorea/...` paths in the unit files (or symlink), then `./jetson.sh --build` .
6. **Clean stale build artifacts** from before the I2C-sensor migration (the old `jetson_sensors` Jetson-I2C package is gone from source but its build output may linger):

```
cd software/ros2_ws && rm -rf build/jetson_sensors install/jetson_sensors
```
7. **Bench-verify the odometry constants** before trusting Nav2 (these affect map

scale + heading): VESC `gear_ratio` (23.333), wheel circumference, track width, and the ZED yaw-gyro sign (`imu_yaw_sign`). See architecture §18 item 12.

Then the normal flow is just `./jetson.sh` (Part 1).

Troubleshooting

First rule: **most "nothing works" problems are the link (Zenoh/domain) or the bridge being down.** Start at §A, then jump to the relevant section.

Diagnostic tools (`./diagnose.sh`)

Before reading the sections below, run the diagnostics — they read the robot's telemetry and tell you what's actually wrong (PPM, CAN presence, I2C sensors, comms). Read-only; run on either machine (needs the link up + `esp32-bridge`):

```
./diagnose.sh          # one-shot health snapshot of everything (~6 s)
./diagnose.sh link     # comms: which boards are talking, flags, e-stop    (live)
./diagnose.sh ppm      # RC/PPM: per-channel bars, ppm_ok, e-stop/vflip    (live)
./diagnose.sh can      # CAN presence: all 6 VESCs + the 6 arm joints      (live)
./diagnose.sh sensors  # arm-PCB I2C: thermal + mag rates / sensor_ok      (live)
```

A missing VESC/joint, a dropped board, a stuck PPM channel, or a disabled sensor shows up immediately with a one-line "why" hint. Lower-level manual probes:

```
ros2 node list          # who's alive on the graph
ros2 topic hz /robot/telemetry # is the chassis PCB talking? (~50 Hz)
./jetson.sh --status    # on the robot: which services are up
journalctl --user -u <unit> -e # last logs of any --user service
```

A. Connection & middleware (robot ↔ laptop)

Laptop sees nothing — dead dashboard LED, `ros2 node list` shows only local nodes.

- **Same domain?** `echo $ROS_DOMAIN_ID` must match (default 20) on Jetson, laptop,

and the Zenoh router. A mismatched domain = total silence with no error.

- **Zenoh router up on both hosts?** `systemctl status rmw-zenoh-router.service`.
Nothing listening on `:7447` (`ss -ltnp | grep 7447`) → re-run
`sudo PEER_IP=<other-host> bash scripts/setup-zenoh.sh`.
- **Right RMW?** a fresh shell must have `RMW_IMPLEMENTATION=rmw_zenoh_cpp` on both hosts. If it's unset/ `rmw_fastrtps_cpp`, `setup-zenoh.sh` never ran there (or you started the process from an old shell — re-source / re-login).
- **Can they reach each other?** `ping 192.168.50.10 / .11`. Wrong/duplicate IP, wrong subnet, or Wi-Fi dropped → fix the static IP (NetworkManager, §0.1).
- **Stale discovery cache:** `ros2 daemon stop`, then re-list.

/robot/* services missing in the GUI Robot Systems window.

- The servers run on the **Jetson**. `./jetson.sh --status` — are `robot-manager` and `map-manager` *active*? If not, `journalctl --user -u robot-manager -e`.
- If managers are up but the laptop can't see them → it's a link problem (§A above).

B. No telemetry / bridge (esp32_bridge)

Dashboard connected but all telemetry is zero / no mode / no battery.

- Is the bridge running? `./jetson.sh --status` ;
`journalctl --user -u esp32-bridge -e`.
- **Serial device not found / not bound.** The bridge auto-binds both ESP32 boards by identity from `/dev/ttyCH341USB*` + `serial/by-id`. Check they enumerate:
`ls -l /dev/ttyCH341USB* /dev/serial/by-id/`. If a board is on a different path, pin it: `./jetson.sh --serial /dev/ttyXXX` (or add its glob to the launch's `serial_candidates`). Loose USB / wrong cable is the usual culprit.
- **Board identity never arrives** (`MSG_BOARD_IDENTITY`): the ESP32 isn't running RoboCorea firmware, is at the wrong baud (must be **921600**), or is held in reset. Reflash / power-cycle the board.
- **Only one board bound** → the other PCB's USB link is down. The chassis PCB carries drive telemetry; the arm PCB carries thermal/mag + arm. Missing telemetry type points at which board.

C. Arm — won't arm, won't move, faults

Arm does nothing when I jog the gamepad.

- It boots **disarmed on purpose** (`ARM_PASSIVE_BOOT`). Click **Arm** on the dashboard (or the "arm now?" prompt at GUI startup). Watch `/arm/state` : it should go `UNINIT → INITIALIZING → READY` . The SDLS servo publishes nothing while disarmed/faulted, so the twin won't move either.
- **Wrong mode. Chassis mode** torque-offs the wrist (J5/J6) so they don't overheat while parked — they won't move. Switch to **Dexterity** for full-arm motion (dashboard toggle).
- **No gamepad.** The arm is jogged from the **workstation gamepad** (`joystick_servo`), not the RC. Is it plugged in and did `laptop.sh` start joy (default on; not `--no-joystick`)? Check `ros2 topic echo /joy` moves, and `/servo_node/delta_twist_cmds` is non-zero while you push a stick.
- **Network dropped.** Joint commands are computed on the laptop and sent down; if the link drops the arm **freezes** (holds) while the base still answers the RC. Fix the link (§A), then re-jog.

`/arm/state` shows **FAULT** , or a joint won't initialize (a CAN-presence dot is dark).

- A **latched CAN fault disarms the arm and rejects motion** — clear it with a fresh **Arm** (Disarm → Arm). `ros2 topic echo /arm/fault / /arm/can_presence` shows which joint(s).
- A dark presence dot = that controller isn't answering on the arm CAN bus:
 - **Bitrate:** every controller must be **500 kbps**. ODrive ships at **250 kbps** — change + save it first, or it never joins the bus.
 - **Wrong CAN ID / not powered / loose CAN wiring.** IDs: ODrive J1=0x10/J2=0x11/J3=0x12, ZE300 J4=id 1, LKTech J5=14/J6=15. (Cross-check on the bench.)
- After a power-cycle or moving a joint by hand, ODrive/LKTech/ZE300 **zero offsets** shift — re-Arm to recapture them before precise moves.

Arm moves the wrong way / overshoots / wrong scale. Gear ratios + direction signs are inherited from the legacy arm and must be bench-verified (ODrive 48:1, LKTech 10:1, ZE300 16384 cpr). Start with tiny commands.

Gripper won't open/close. RT/LT on the gamepad → `/gripper` (rate). It's on the

arm PCB (PWM, `GPI026`), not the CAN bus, and only in the **ARM** board build. Check `ros2 topic echo /gripper moves`; confirm the servo is on the **11.1–15 V** rail (not the ESP32) with a common ground. An **E-stop freezes it** at its current angle.

D. Drive, flippers & RC

Tracks don't respond to the sticks.

- **E-stop active.** Ch6 **down** = e-stop (mode shows `ESTOP`); return Ch6 to centre. A **GUI software e-stop** also latches — clear it on the dashboard.
- **RC link down.** The FS-iA6B must be bound to your transmitter and powered; PPM on the chassis ESP32 (`GPI04`). No valid PPM within `PPM_TIMEOUT_MS` → tracks stop, flippers hold (failsafe). `ros2 topic echo /robot/ppm` should change as you move the sticks; `/robot/flags` bit `ppm_ok` should be set.
- **VESC not on the bus.** Traction VESCs must be **500 kbps** with the right CAN IDs (L=60, R=50). `ros2 topic echo /motors/vesc_status` should list all six.

Flippers drift, jump, or don't hold.

- The flipper position loop runs **on the VESC** (LispBM) — the four flipper VESCs (20/10/40/30) must have `flipper_position.lisp` flashed ("Run at startup"). Without it they won't position-hold.
- On (re)entering flipper control the ESP seeds the target from the VESC-reported angle (bumpless). A jump usually means the report frame (`0x7F`) isn't coming back.

Robot drives "backwards" / steering mirrored. Ch6 is **up** = **virtual-flip** ("drive from the other end"): the odometry panel shows **REVERSE**. Return Ch6 to centre.

E. Sensors (ZED, lidar, thermal/mag)

Start ZED+Lidar fails or the LED stays red.

- `journalctl --user -u zed.service -e / -u lidar.service -e` .
- **ZED:** needs **USB 3** bandwidth; wrong `camera_model` (default `zed2` — edit `zed.service`); ZED SDK / udev not installed. `ros2 topic hz /zed/zed_node/odom` .
- **Lidar:** `/dev/rplidar` missing → install the udev rule (§0.3); wrong port; cable.

```
ros2 topic hz /scan .
```

Thermal / magnetometer readout dead. They are on the **arm PCB** now (not the Jetson) — so they need the **arm board's USB link up** (§B) and the dashboard

Thermal/Mag enable toggles on (`/sensors/enable_mask`).

```
ros2 topic hz /sensors/thermal / /sensors/mag .
```

 No Jetson I2C is involved.

F. Mapping / SLAM

Start SLAM fails immediately. Start **Sensors** first — SLAM needs `/scan` + ZED odom. `journalctl --user -u rescue-mapping.service -e` .

Map looks smeared / walls double up.

- Drive **slower**; revisit a known spot to force re-registration; avoid fast spins.
- **Carrying the robot / bench with no tracks?** The wheel-fusion EKF mis-models that — use `./jetson.sh --no-ekf` (ZED-only odom) or the live toggle `systemctl --user set-environment ROBOCOREA_USE_EKF=false` then restart SLAM.

- Verify the TF tree has exactly **one publisher per edge**

(`map→odom→base_footprint→base_link→base_laser`):

```
ros2 run tf2_tools view_frames .
```

Two publishers on `odom→base_footprint` = the ZED wrapper is still publishing TF (must be `publish_tf:=false` , which `zed.service` sets) or both EKF and ZED-planar are on.

/map not updating. `ros2 topic hz /map` ; check `slam_toolbox` is alive

(`ros2 node list | grep slam`) and `/scan_flat` is publishing in the `base_laser` frame.

G. Localization & Navigation

Loaded a map but the robot isn't where it should be on the map. AMCL starts from your **Set Start Pose** click — place it accurately at the robot's real location + heading. The particle cloud should tighten; if it diverges, re-set it, or drive a little so scan-matching converges.

Navigation won't start (pre-flight blocked). The `preflight:` line under the Navigation stack names what's failing:

- `scan` — sensors not started, or lidar dead (`ros2 topic hz /scan_flat`).
- `odom` — the EKF isn't publishing (`ros2 topic hz /odometry/filtered`); often no `/odom/wheel` from the bridge (bridge down → §B, or run `--no-ekf` on the bench).
- `tf` — a missing edge: start pose not set, or Localization not up. `view_frames` .
- `map` — no map loaded (Maps tab → **Load** first; **Refresh** if it's not listed).

Nav2 plans but the robot doesn't move.

- **AUTO DRIVE** is off (dashboard) — that's the autonomy gate. Turn it on.
- An **RC drive stick is deflected** or **virtual-flip** is engaged — both **latch AUTO DRIVE off**. Centre the sticks, then re-enable AUTO DRIVE (a nudge won't silently resume it).
- `ros2 topic echo /cmd_vel` should be non-zero while navigating, `/autonomy/state true` , and something must subscribe `/cmd_vel` (the bridge — §B).

Robot oscillates, stalls, or can't reach the goal. Goal outside the mapped/free area; localization not converged (above); costmap inflation/footprint vs. the real robot. Watch the global/local plans in RViz/the map window; re-send the goal.

H. Video (C920 / RF cams)

No C920 video in the GUI. `journalctl --user -u c920-stream.service -e` . The streamer is pinned to `/dev/video2` port `8890` — confirm the camera enumerates there (`v4l2-ctl --list-devices`) and the GUI's SRT settings point at the Jetson IP

- that port. SRT is separate from ROS, so the link being healthy for ROS doesn't guarantee the video port is reachable (check a firewall).

No RF driving-cam video. Those are local USB digitizers on the **laptop** (`/dev/videoN`), nothing to do with the robot/ROS — check the digitizer + the GUI source selection.

I. Build, services & GUI

colcon build fails after I edited a unit / added a file. `--symlink-install` can leave a stale symlink — `rm -rf build/<pkg> install/<pkg>` and rebuild that package.

A `--user` service "is active" but dies when I close the SSH session. Linger isn't enabled: `sudo loginctl enable-linger $USER` ([jetson.sh](#) does this).

A unit fails on start. `journalctl --user -u <unit> -e`. Common: the checkout isn't at `~/RoboCorea` (fix the `%h/RoboCorea/...` path in the unit), or the workspace isn't built (`./jetson.sh --build`).

GUI won't launch. Missing build deps (Qt6 incl. OpenGL, OpenCV w/ GStreamer, GStreamer SRT/Opus plugins, ONNX Runtime, Assimp, ZBar, `ros-humble-urdf`) — see architecture §17 + the `gui` README. Rebuild: `./laptop.sh --build`.

Clean teardown / orphans

Everything runs under systemd cgroups, so stop is deterministic (no orphaned nodes):

```
./jetson.sh --stop                # stop all RoboCorea services
journalctl --user -u esp32-bridge -u robot-manager -u map-manager -f  # tail logs
```

Quick reference

ROBOT (Jetson 192.168.50.10)	LAPTOP (192.168.50.11)
<code>./jetson.sh</code> bring up	<code>./laptop.sh</code> GUI + arm + twin
<code>./jetson.sh --status</code> what's running	<code>./laptop.sh --build</code> rebuild first
<code>./jetson.sh --stop</code> stop all	GUI $\boxtimes$ Robot Systems perception + maps
<code>./diagnose.sh [link ppm can sensors]</code>	health checks (either machine)

Map: Sensors ► → Mapping ► → drive (RC) → 2D Map: Save Map...

Nav: Sensors ► → Maps: Load → 2D Map: Set Start Pose → (preflight green)
→ Navigation ► → dashboard AUTO DRIVE on → send goal

Same session (just mapped): Stop Mapping → Maps: Load (same map) → Set Start Pose → Start Nav. "Load" = start localization; mapping mode $\neq$ nav.

Stop: Ch6 down (RC e-stop) | GUI e-stop | RC stick nudge = drop autonomy

Maps live on the Jetson: `~/maps/<name>/` (persist across reboots)

See `reference/architecture.md` for the full design,

and `software/ros2_ws/src/rescue_nav/docs/COMPETITION_WORKFLOWS.md` for the launch-file-level (non-GUI) workflow.